When Should Personal Evidence Become Model Weights?

A Code-Grounded Method and Novelty Audit for Per-User Offline-to-Online Personalization

Technical Design Note

August 29, 2026

Abstract

This note asks a narrow question: when should evidence about one user remain in an editable prompt or memory, and when should it update that user's model adapter? The question matters because a parameter update is slower, harder to inspect, and more likely to affect unrelated tasks. It is useful only when it improves future behavior beyond what the same evidence can achieve as context.

Two local codebases already contain real training. One trains a Qwen3-4B LoRA adapter with DPO. The other implements GRPO and online policy distillation (OPD). These mechanisms are not new contributions. Prior work also covers per-user adapters, continual adapter updates, retrieval memory, and context-conditioned steering. The closest method, SPRInG, already combines a user adapter with retrieved history.

The remaining hypothesis is a paired placement test. For the same evidence, build one context candidate and one adapter candidate. Compare them on later tasks, then activate the adapter only if it beats the context candidate while preserving task success and safety. This is a proposed experiment, not a verified result or a settled novelty claim.

1 Decision in Plain Language

A prompt is the default place for new personal information. It is easy to read, change, delete, and limit to one task. A user adapter is worth testing only when a preference repeats across sessions, remains stable, applies to several tasks, and should work without profile text at runtime.

Learning $\Delta\theta_u$ is not useful merely because the code can train it. The update must answer a harder question:

> Does the adapter improve later tasks more than giving the same evidence to the model as prompt or memory, without reducing task success or safety?

If the answer is no or remains uncertain, the evidence should stay in editable context.

2 Terms Used in This Note

Post-deployment online update. In this note, an online update occurs after the system has started serving the user. New user evidence arrives, an optimizer changes parameters, and the system produces a new loadable checkpoint. Training on a fixed dataset with fresh model rollouts is often called online RL, but it is not the same as learning from a deployed user's later interactions.

Personalized agent. A personalized agent completes tasks through tools, memory, actions, or clarifying questions while adapting to one user. A system that only changes the style or content of generated text is personalized generation, not necessarily personalized agent execution.

Table 1: Four objects that should not be treated as the same thing.

Object	Meaning	Used when	Main property
Profile prompt	A short text statement of user facts or preferences	Added directly to the input	Easy to inspect and edit, but consumes context
Retrieved memory M_u	Selected records from the user's earlier interactions	Retrieved for a relevant request	Keeps details outside the model, but retrieval can miss or add noise
Training hint h	A correction shown to a teacher during training	Used to make a target or distillation signal	A supervision channel, not necessarily deployed user state
User adapter $\Delta\theta_u$	A learned parameter change for user u	Loaded with the frozen base model	Works without profile text, but is harder to inspect and undo

3 What the Local Code Already Implements

The two repositories contain training algorithms, not only data generation.

Table 2: Code-grounded status of the two local implementations.

Repository	Implemented update	Current evidence	Main defect
`/fsx/xzhichao/workspace/offlinetraining`	Qwen3-4B LoRA trained with `TRL DPOTrainer`	A clean comparison reports 48.69% macro and 49.12% micro win rate, with sign-test $p = 0.726$	The profile is extracted before the holdout split, synthetic pairs are assumed to be preferences, and there is no adapter-only test arm
`/fsx/xzhichao/workspace/oocso`	GRPO, hint-conditioned OPD, replay behavior cloning, and optimizer updates	The current three-user run is incomplete; an older result file uses a different configuration	The present offline phase uses scalar-reward GRPO rather than OPD, and no complete current run establishes an offline-to-online gain

The DPO result does not show a reliable training gain. It also does not prove that user adapters cannot work. The comparison tested trained-plus-profile against base-plus-profile, so it did not isolate what the adapter learned.

OOCSO already implements "teacher sees a hint, student does not." A future paper can use OPD as a training backend, but cannot present that mechanism as new.

4 Why Not Use a Prompt for Everything?

4.1 Cases where prompt or memory is better

Keep evidence outside the weights when it is:

- observed once or weakly supported;

- temporary, such as a tone request for one report;
- tied to one task, project, or tool;
- likely to change;
- easy to state as a short rule; or
- sensitive enough that the user should be able to inspect and delete it.

This choice has real costs. Retrieval can select the wrong record. Long profiles consume context and are processed again for every request. The model may also ignore a relevant instruction.

4.2 Cases where an adapter may help

Test an adapter when the preference:

- appears in several independent sessions;
- remains stable over time;
- affects several task families;
- cannot be expressed well as one short instruction; and
- should affect behavior even when profile text is absent.

An adapter may reduce repeated prompt cost and make the behavior more consistent. It also introduces training cost, forgetting risk, broader behavior changes, and rollback work. These costs make $\Delta\theta_u$ an empirical choice rather than a default.

5 Closest Prior Work

5.1 OPPU and Personalized DPO

OPPU trains one PEFT module from each user's historical behavior [1]. It can also use retrieved history or a generated profile. It is an offline method. It does not decide, for each later observation, whether that observation should remain in memory or enter the adapter.

Personalized RLHF, evaluated through personalized DPO, learns user-conditioned representations and LoRA parameters from offline preference pairs [2]. It does not study chronological post-deployment updates, retrieval memory, or agent task outcomes.

5.2 SPRInG

SPRInG is the closest personalization method [3]. It already contains:

- one evolving LoRA adapter per user;
- likelihood-based selection of incoming interactions;
- a bounded buffer for examples that remain hard after an update;
- relevance-gated retrieval; and
- token-level interpolation between adapter-only and retrieval-conditioned generation.

Thus, "combine memory and an adapter" is not new. SPRInG chooses training examples through likelihood drift, then retains post-update residuals. It does not train matched context and adapter candidates from the same evidence and select between them using later task success and safety.

SPRInG simulates sequential updates with chronological LongLaMP records. It personalizes text generation. It does not execute tool-based agent tasks or learn from live post-deployment agent outcomes.

5.3 EvolveR

EvolveR studies an agent-level experience loop rather than per-user personalization [4]. It stores distilled principles as retrieved context and updates a global policy with GRPO. Its standard method masks retrieved experience tokens from the training loss.

EvolveR also tests direct experience absorption by unmasking those tokens. Its reported average drops from 0.382 to 0.371. The authors attribute the drop to noisy or irrelevant retrieved principles. This result supports a conservative design: context should not be written into weights without a quality and relevance test.

5.4 CoSteer

CoSteer is a tuning-free alternative [5]. A local small model runs with and without private context. The difference between its two token distributions steers a frozen cloud model. CoSteer does not learn $\Delta\theta_u$. It shows that runtime personalization can be stronger than plain prompting without changing model weights, so it is a useful baseline when privacy and local inference matter.

6 Novelty Audit

Table 3: What the closest methods already cover. "Sequential" means that later records can change model parameters.

Method	Learned object	External user state	Sequential update	Agent task outcomes
OPPU	Per-user PEFT	Optional profile or retrieval	No	No
Personalized DPO	Shared LoRA and user representation	Optional user text	No	No
SPRInG	Per-user LoRA	Residual retrieval buffer	Yes, on a historical stream	No
EvolveR	Global agent policy	Experience Base	Yes, during agent training	Yes, but not per user
CoSteer	None	Private local context	No parameter update	No
OOCSO / OPD	Per-user LoRA	Training hints	Implemented in simulation	Simulated rewards
Proposed test	Per-user adapter when accepted	Editable memory otherwise	Yes, after deployment	Required

What is not new. Per-user LoRA, personalized DPO, continual adapter updates, retrieval plus parameters, hint distillation, and context-versus-parameter ablations all exist.

What may remain distinct. The tentative difference is an evidence-level, matched comparison. The same evidence creates a context candidate and an adapter candidate from the same parent checkpoint. Later chronological tasks decide where the evidence should live. The adapter is accepted only when its advantage over context is large enough and task and safety scores do not regress.

Table 4: The narrow mechanism-level difference under study.

Method	How evidence enters weights	How evidence remains external	How an update is accepted
SPRInG	High likelihood-drift examples train the adapter	Hard post-update residuals enter the retrieval buffer	Fixed selection rule; no matched context-versus-adapter validation
EvolveR	Agent trajectories update the policy; experience-token absorption is an ablation	Distilled principles are retrieved as context	Training reward; no per-evidence storage decision
OOCSO / OPD	The no-hint student learns from a hint-conditioned teacher	Hints supervise training but are not a matched deployed memory arm	Reward-driven update; no context-versus-adapter activation test
Proposed test	One candidate adapter is trained from eligible evidence	A matched candidate stores the same evidence as context	Later tasks must favor the adapter and pass task and safety constraints

Current verdict. This is a defensible research hypothesis, not yet a publication novelty claim. The claim becomes credible only after a broader search confirms that no prior method performs this paired placement and constrained activation procedure.

7 Proposed Paired Placement Test

7.1 State and evidence

For user u at time t, the active state is

$$S_u^{(t)} = \left(\theta_0, \Delta\theta_u^{(t)}, M_u^{(t)}, c_t\right), \tag{1}$$

where c_t identifies the active checkpoint. The frozen base parameters are θ_0, the user adapter is $\Delta\theta_u^{(t)}$, and the editable memory is $M_u^{(t)}$.

Each evidence record is

$$e_i = (u, t_i, x_i, \tau_i, y_i, f_i, o_i, p_i). \tag{2}$$

Here x_i is the task, τ_i is the agent trajectory, y_i is the final answer, f_i is user feedback, o_i is the environment outcome, and p_i records provenance. Evidence cannot train a model when the user, task, outcome, or generating checkpoint is unknown.

7.2 Matched candidates

Let E_t be a checked batch of new evidence. Build both candidates from the same active state.

Context candidate.

$$M_{u,\mathrm{ctx}} = \mathrm{MemoryUpdate}\left(M_u^{(t)}, E_t\right), \qquad \Delta\theta_{u,\mathrm{ctx}} = \Delta\theta_u^{(t)}. \tag{3}$$

Adapter candidate.

$$\Delta\theta_{u,\mathrm{par}} = \mathrm{TrainAdapter}\left(\Delta\theta_u^{(t)}, E_t\right), \qquad M_{u,\mathrm{par}} = M_u^{(t)}. \tag{4}$$

The training backend may be SFT, DPO, or OPD. These are inherited components. The adapter candidate is evaluated without adding the new evidence to its prompt.

7.3 Activation rule

Let V_t contain sessions that occur after E_t but before the untouched final test period. Evaluate the context candidate m_{ctx}, the adapter candidate m_{par}, and the current model m_0 under the same decoding settings.

Define the paired personalization difference

$$D_{\mathrm{pref}} = U_{\mathrm{pref}}(m_{\mathrm{par}}; V_t) - U_{\mathrm{pref}}(m_{\mathrm{ctx}}; V_t). \tag{5}$$

Activate the adapter only if

$$\mathrm{LCB}(D_{\mathrm{pref}}) > \delta, \tag{6}$$

$$\mathrm{LCB}(U_{\mathrm{task}}(m_{\mathrm{par}}; V_t) - U_{\mathrm{task}}(m_0; V_t)) \geq -\epsilon, \tag{7}$$

$$U_{\mathrm{safe}}(m_{\mathrm{par}}; V_t) \geq U_{\mathrm{safe}}(m_0; V_t). \tag{8}$$

The lower confidence bound (LCB) prevents a small noisy difference from triggering an update. Equation (7) is a non-inferiority test: task success may fall by at most the predeclared margin ϵ.

If the adapter fails, use context only when context improves personalization without reducing task or safety scores. Otherwise, keep the evidence pending, ask the user for clarification, or make no change. The procedure never forces every record into memory or weights.

Algorithm 1: Paired evidence placement and conservative activation

Input: Active state $S_u^{(t)}$, new evidence E_t, later validation window V_t, thresholds δ, ϵ
Output: Versioned next state $S_u^{(t+1)}$

```
Check user identity, time, task outcome, feedback, and checkpoint provenance;
Remove unsupported, contradictory, unsafe, or task-failing records;
if eligible evidence is insufficient then
    keep the active state and record a pending decision;
    return S_u^(t);
Build m_ctx by storing E_t as editable context;
Build m_par by training a child adapter on the same E_t;
Evaluate m_0, m_ctx, and m_par on the same V_t;
if Eqs. (6) to (8) pass then
    activate the child adapter and save m_0 as the rollback target;
else
    if the context candidate improves personalization and passes task and safety checks then
        activate the context candidate;
    else
        make no change or ask the user to clarify;
Write the data boundary, metrics, parent, child, and decision to the ledger;
return the versioned next state;
```

8 Experiment That Can Answer the Question

8.1 Chronological data boundary

Split whole sessions before extracting a profile or generating training data:

$$H_u^{\text{past}} \prec E_t \prec V_t \prec H_u^{\text{test}}. \tag{9}$$

The order symbol means "occurs earlier than." The profile, memory, hints, preference pairs, and adapter may use only the records available at that time. The final test period remains untouched until the method and thresholds are fixed.

8.2 Required arms

Table 5: Minimum comparison needed to identify where personalization comes from.

Arm	Question answered
Frozen base	What happens without personalization?
Base plus profile or memory	How much can explicit context do without training?
User adapter only	Did the weights store useful personal behavior?
User adapter plus memory	Are the two representations complementary?
OPPU-style adapter	Does direct offline per-user training suffice?
SPRInG	Does likelihood-based selective adaptation plus retrieval suffice?
OOCSO-style OPD	Does hint distillation improve later no-hint behavior?
CoSteer, when deployment permits	Can tuning-free local steering avoid a parameter update?
Paired placement test	Does evidence-level placement improve over fixed storage rules?

8.3 Separate outcomes

Do not combine all outcomes into one score. Report:

- preference match on future sessions;
- environment-verified task success;
- safety violations and invalid actions;
- unrelated-task regression;
- adaptation speed after a real preference change;
- forgetting after later updates;
- prompt tokens, training cost, and inference cost;
- update count, rejection count, and rollback count; and
- uncertainty intervals over users and tasks.

Pairwise judges must allow ties and invalid outputs. A style judge cannot replace task correctness. A result from one synthetic user cannot establish a general personalization method.

9 Implementation Order

The shortest credible path is:

1. Split the offline DPO conversations by time before profile extraction.
2. Add base, base-plus-profile, adapter-only, and adapter-plus-profile evaluation arms.
3. Verify every synthetic pair with separate preference, task, and safety checks.
4. Reproduce one clean offline result.
5. Finish one current OOCSO run with fixed configuration and complete checkpoint lineage.
6. Add the evidence ledger and the two matched candidates around the existing SFT, DPO, or OPD backend.
7. Test placement on controlled stable, temporary, task-local, contradictory, and drifting preferences.
8. Move to real tool-using agent trajectories only after the storage decision works on the smaller test.

10 Final Assessment

The local projects already contain DPO, GRPO, OPD, LoRA updates, and evaluation drivers. They are not data-generation-only projects. Their current results do not show that a user adapter reliably improves future behavior.

There is also no general reason to prefer $\Delta\theta_u$ over a prompt. Prompt or memory is better for recent, uncertain, temporary, task-local, changing, or sensitive evidence. An adapter is worth the cost only for repeated and stable behavior that transfers across tasks and works without runtime profile text.

The proposed paired placement test is narrower than "memory plus adapter." It asks both representations to compete using the same evidence and later tasks. SPRInG, EvolveR, and the local OPD implementation make this a plausible next experiment, but they also narrow the novelty claim. The paper should claim a new method only after the literature audit and matched experiment both support that claim.

References

[1] Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning. In *EMNLP*, 2024. https://aclanthology.org/2024.emnlp-main.372/.

[2] Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Leqi Liu. Personalized Language Modeling from Personalized Human Feedback. arXiv:2402.05133, 2024. https://arxiv.org/abs/2402.05133.

[3] Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM Personalization via Selective Parametric Adaptation and Retrieval-Interpolated Generation. arXiv:2601.09974, 2026. https://arxiv.org/abs/2601.09974.

[4] Rong Wu, Xiaoman Wang, Jianbiao Mei, and others. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle. arXiv:2510.16079, 2025. https://arxiv.org/abs/2510.16079.

[5] Hang Lv, Sheng Liang, Hao Wang, and others. CoSteer: Collaborative Decoding-Time Personalization via Local Delta Steering. arXiv:2507.04756, 2025. https://arxiv.org/abs/2507.04756.